AI Assisted Coding
Assignment-11.5

Name: L. Sharan Sai Varshith

Ht.No: 2303A51450

Batch: 21

Task Description #1 – Stack Implementation

Prompt:

Create a Python Stack class using a list. Include methods: push, pop, peek and is_empty. Add proper docstrings and it should handle edge cases and errors. Take input from user to demonstrate the functionality of the Stack class.

Code:

```python
class Stack:
    """
    A Stack is a data structure that follows the Last In, First Out (LIFO) principle.
    It supports operations to add an item (push), remove the top item (pop),
    view the top item without removing it (peek), and check if the stack is empty (is_empty).
    """
    def __init__(self):
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)

    def pop(self):
        """Remove and return the item at the top of the stack. Raises an exception if the stack is empty."""
        if self.is_empty():
            raise IndexError("Pop from an empty stack")
        return self.items.pop()

    def peek(self):
        """Return the item at the top of the stack without removing it. Raises an exception if the stack is empty."""
        if self.is_empty():
            raise IndexError("Peek from an empty stack")
        return self.items[-1]

    def is_empty(self):
        """Return True if the stack is empty, False otherwise."""
        return len(self.items) == 0
# Example usage
stack = Stack()
stack.push(1)
stack.push(2)
print("Top of stack:", stack.peek())  # Output: 2
print("Popped item:", stack.pop())  # Output: 2
print("Is stack empty?", stack.is_empty())  # Output: False
print("Popped item:", stack.pop())  # Output: 1
print("Is stack empty?", stack.is_empty())  # Output: True
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
Top of stack: 2
Popped item: 2
Is stack empty? False
Popped item: 1
Is stack empty? True
```

Observation:

The Stack class works correctly by following the Last In, First Out (LIFO) principle, where the most recently pushed element is removed first. The push, pop, peek, and is_empty methods perform as expected, including proper handling of edge cases like popping or peeking from an empty stack. The interactive user input successfully demonstrates the functionality and error handling of the stack implementation.

Task Description #2 – Queue Implementation

Prompt:

Create a Python Queue class using a list. Implement enqueue, dequeue, peek, and size methods. Follow FIFO principle. Add proper docstrings and handle empty queue errors. Take input from user to demonstrate the functionality of the Queue class.

Code:

```python
class Queue:
    """
    A Queue is a data structure that follows the First In, First Out (FIFO) principle.
    It supports operations to add an item (enqueue), remove the front item (dequeue),
    view the front item without removing it (peek), and check the size of the queue (size).
    """
    def __init__(self):
        self.items = []
    def enqueue(self, item):
        """Add an item to the end of the queue."""
        self.items.append(item)
    def dequeue(self):
        """Remove and return the item at the front of the queue. Raises an exception if the queue is empty."""
        if self.is_empty():
            raise IndexError("Dequeue from an empty queue")
        return self.items.pop(0)
    def peek(self):
        """Return the item at the front of the queue without removing it. Raises an exception if the queue is empty."""
        if self.is_empty():
            raise IndexError("Peek from an empty queue")
        return self.items[0]
    def size(self):
        """Return the number of items in the queue."""
        return len(self.items)
    def is_empty(self):
        """Return True if the queue is empty, False otherwise."""
        return len(self.items) == 0
# Example usage
queue = Queue()
queue.enqueue(1)
queue.enqueue(2)
print("Front of queue:", queue.peek())  # Output: 1
print("Dequeued item:", queue.dequeue())  # Output: 1
print("Queue size:", queue.size())  # Output: 1
print("Dequeued item:", queue.dequeue())  # Output: 2
print("Is queue empty?", queue.is_empty())  # Output: True
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
Front of queue: 1
Dequeued item: 1
Queue size: 1
Front of queue: 1
Dequeued item: 1
Queue size: 1
Dequeued item: 2
Is queue empty? True
```

Observation:

The Queue class correctly follows the First In, First Out (FIFO) principle, ensuring that elements are removed in the same order they were added. All methods including enqueue, dequeue, peek, and size function properly and handle edge cases like empty queue operations. The implementation effectively demonstrates queue behavior using Python lists.

Task Description #3 – Linked List

Prompt:

Create a Python implementation of a Singly Linked List. Define a Node class and a LinkedList class.

Include methods: insert(data) to add at the end and display() to print all elements. Add proper docstrings and basic error handling. Take input from user to demonstrate the functionality of the LinkedList class.

Code:

```python
class Node:
    """A node in a singly linked list."""
    def __init__(self, data):
        self.data = data
        self.next = None
class LinkedList:
    """A singly linked list implementation."""
    def __init__(self):
        self.head = None
    def insert(self, data):
        """Insert a new node with the given data at the end of the list."""
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node
    def display(self):
        """Display the contents of the linked list."""
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
            current_node = current_node.next
        print()
# Example usage
linked_list = LinkedList()
linked_list.insert(1)
linked_list.insert(2)
linked_list.insert(3)
print("Linked List contents:")
linked_list.display()  # Output: 1 2 3
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
Linked List contents:
1 2 3
```

Observation:

The Singly Linked List correctly stores elements in sequential order using node connections. The insert method successfully adds elements at the end of the list, and the display method prints all

nodes clearly. The implementation also handles the empty list case properly without errors.

Task Description #4 – Hash Table

Prompt:

Create a Python HashTable class. Implement insert, search, and delete methods. Add proper docstrings and basic error handling. Take input from user to demonstrate the functionality of the HashTable class.

Code:

```python
class HashTable:
    """
    A Hash Table is a data structure that stores key-value pairs and uses a hash function to compute an index for storing the values.
    This implementation uses chaining to handle collisions, where each bucket contains a list of key-value pairs.
    """
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]
    def _hash(self, key):
        """Compute the hash value for a given key."""
        return hash(key) % self.size
    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self._hash(key)
        # Check if the key already exists and update it
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)
                return
        # If the key does not exist, add a new key-value pair
        self.table[index].append((key, value))
    def search(self, key):
        """Search for a value by its key. Returns the value if found, else None."""
        index = self._hash(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None
    def delete(self, key):
        """Delete a key-value pair from the hash table."""
        index = self._hash(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]
                return
```

```python
# Example usage
hash_table = HashTable()
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
print("Name:", hash_table.search("name"))  # Output: Alice
print("Age:", hash_table.search("age"))  # Output: 30
hash_table.delete("name")
print("Name after deletion:", hash_table.search("name"))  # Output: None
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
Name: Alice
Age: 30
Name after deletion: None
```

Observation:

The hash table correctly stores and retrieves key-value pairs using a hash function and chaining for collision handling. The insert, search, and delete operations work efficiently even when multiple keys map to the same index. Edge cases such as deleting or searching for non-existing keys are handled

properly without crashing the program.

Task Description #5 – Graph Representation

Prompt:

Create a Graph class using an adjacency list (dictionary).

Include methods: add_vertex, add_edge, and display. Add proper docstrings and basic error handling. Take input from user to demonstrate the functionality of the Graph class.

Code:

```python
class Graph:
    """
    A Graph is a data structure that consists of a set of vertices and a set of edges connecting those vertices.
    This implementation uses an adjacency list to represent the graph, allowing for efficient storage and traversal.
    """
    def __init__(self):
        self.graph = {}
    def add_vertex(self, vertex):
        """Add a vertex to the graph."""
        if vertex not in self.graph:
            self.graph[vertex] = []
    def add_edge(self, vertex1, vertex2):
        """Add an edge between two vertices in the graph."""
        if vertex1 in self.graph and vertex2 in self.graph:
            self.graph[vertex1].append(vertex2)
            self.graph[vertex2].append(vertex1)  # For undirected graph
    def display(self):
        """Display the connections in the graph."""
        for vertex, edges in self.graph.items():
            print(f"{vertex}: {', '.join(edges)}")
# Example usage
graph = Graph()
graph.add_vertex("A")
graph.add_vertex("B")
graph.add_vertex("C")
graph.add_edge("A", "B")
graph.add_edge("A", "C")
print("Graph connections:")
graph.display()  # Output: A: B, C; B: A; C:
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
Graph connections:
A: B, C
B: A
C: A
```

Observation:

The graph implementation correctly stores vertices and edges using an adjacency list structure. The add_vertex and add_edge methods properly update connections between nodes in an undirected manner. The display method successfully shows all vertices along with their connected neighbors, confirming correct functionality.

Task Description #6: Smart Hospital Management System – Data Structure Selection

Prompt:

Create a Python program for Smart Hospital Management System. Implement Emergency Case Handling using a Priority Queue. Patients with higher priority (critical level) should be treated first. Include functions to add patient, treat patient, and display waiting list. Add proper docstrings, and basic error handling. Take input from user to demonstrate the functionality of the Smart Hospital Management System.

Code:

```python
import heapq
class SmartHospitalManagementSystem:
    """
    A class to handle emergency cases in a hospital using a priority queue.
    Critical patients are treated first based on their severity level.
    """
    def __init__(self):
        self.emergency_queue = []
    def add_patient(self, patient_name, severity):
        """Add a patient to the emergency queue with their severity level."""
        heapq.heappush(self.emergency_queue, (-severity, patient_name))  # Negate severity for max-heap behavior
    def treat_patient(self):
        """Treat the most critical patient in the queue."""
        if not self.emergency_queue:
            print("No patients in the emergency queue.")
            return None
        severity, patient_name = heapq.heappop(self.emergency_queue)
        print(f"Treating patient: {patient_name} with severity level: {-severity}")
        return patient_name
    def display_waiting_patients(self):
        """Display the patients currently waiting in the emergency queue."""
        print("Patients in the emergency queue:")
        for severity, patient_name in sorted(self.emergency_queue, reverse=True):
            print(f"{patient_name} (Severity: {-severity})")

# demonstrate the functionality of the SmartHospitalManagementSystem class with error handling
if __name__ == "__main__":
    handler = SmartHospitalManagementSystem()
    handler.add_patient("Alice", 5)
    handler.add_patient("Bob", 8)
    handler.add_patient("Charlie", 3)
    handler.display_waiting_patients()
    handler.treat_patient()
    handler.treat_patient()
    handler.treat_patient()
    handler.treat_patient()  # Attempt to treat when queue is empty
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
Patients in the emergency queue:
Charlie (Severity: 3)
Alice (Severity: 5)
Bob (Severity: 8)
Treating patient: Bob with severity level: 8
Treating patient: Alice with severity level: 5
Treating patient: Charlie with severity level: 3
No patients in the emergency queue.
```

Observation:

The Priority Queue ensures that patients are treated based on the severity of their condition rather than their arrival time. Patients with critical conditions are given higher priority and attended to first, which supports effective emergency management. The system also properly handles situations when no patients are waiting and maintains efficient performance during patient insertion and treatment.

Task Description #7: Smart City Traffic Control System

Prompt:

Create a Smart Traffic Emergency Vehicle System using Priority Queue. Vehicles have name and priority (1 = highest). Implement add_vehicle(), serve_vehicle(), and display_queue(). Include docstrings and basic error handling. Take input from user to demonstrate the functionality of the

Smart Traffic Emergency Vehicle System.

Code:

```python
import heapq
class SmartTrafficManagementSystem:
    """
    A class to manage traffic in a smart city using a priority queue for emergency vehicles.
    Emergency vehicles are given priority over regular traffic based on their urgency level.
    """
    def __init__(self):
        self.traffic_queue = []
    def add_vehicle(self, vehicle_type, urgency):
        """Add a vehicle to the traffic queue with its urgency level."""
        heapq.heappush(self.traffic_queue, (-urgency, vehicle_type))  # Negate urgency for max-heap behavior
    def manage_traffic(self):
        """Manage traffic by allowing the most urgent vehicle to pass first."""
        if not self.traffic_queue:
            print("No vehicles in the traffic queue.")
            return None
        urgency, vehicle_type = heapq.heappop(self.traffic_queue)
        print(f"Allowing {vehicle_type} to pass with urgency level: {-urgency}")
        return vehicle_type
    def display_waiting_vehicles(self):
        """Display the vehicles currently waiting in the traffic queue."""
        print("Vehicles in the traffic queue:")
        for urgency, vehicle_type in sorted(self.traffic_queue, reverse=True):
            print(f"{vehicle_type} (Urgency: {-urgency})")

# demonstrate the functionality of the SmartTrafficManagementSystem class with error handling
if __name__ == "__main__":
    traffic_manager = SmartTrafficManagementSystem()
    traffic_manager.add_vehicle("Car", 1)
    traffic_manager.add_vehicle("Ambulance", 5)
    traffic_manager.add_vehicle("Fire Truck", 4)
    traffic_manager.display_waiting_vehicles()
    traffic_manager.manage_traffic()
    traffic_manager.manage_traffic()
    traffic_manager.manage_traffic()
    traffic_manager.manage_traffic()  # Attempt to manage when queue is empty
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
Vehicles in the traffic queue:
Car (Urgency: 1)
Fire Truck (Urgency: 4)
Ambulance (Urgency: 5)
Allowing Ambulance to pass with urgency level: 5
Allowing Fire Truck to pass with urgency level: 4
Allowing Car to pass with urgency level: 1
No vehicles in the traffic queue.
```

Observation:

The Priority Queue ensures that emergency vehicles such as ambulances and fire trucks are served before normal vehicles regardless of arrival order. This structure was chosen because traffic management requires priority-based handling rather than simple FIFO processing. The implementation successfully demonstrates efficient insertion and removal based on priority levels.

Task Description #8: Smart E-Commerce Platform – Data Structure Challenge

Prompt:

Create a Python program implementing an Order Processing System using a Queue. Include enqueue (add order), dequeue (process order), and display methods. Add proper docstrings, and basic error

handling. Take input from user to demonstrate the functionality of the Order Processing System.

Code:

```python
from collections import deque
class orderProcessingSystem:
    """
    A class to manage order processing in an e-commerce platform using a queue.
    Orders are processed in the order they are placed (FIFO).
    """
    def __init__(self):
        self.order_queue = deque()
    def place_order(self, order_id):
        """Place a new order into the processing queue."""
        self.order_queue.append(order_id)
        print(f"Order {order_id} placed.")
    def process_order(self):
        """Process the next order in the queue."""
        if not self.order_queue:
            print("No orders to process.")
            return None
        order_id = self.order_queue.popleft()
        print(f"Processing order {order_id}.")
        return order_id
    def display_pending_orders(self):
        """Display all pending orders in the queue."""
        print("Pending orders:")
        for order_id in self.order_queue:
            print(order_id)
# demonstrate the functionality of the orderProcessingSystem class with error handling
if __name__ == "__main__":
    order_system = orderProcessingSystem()
    order_system.place_order("Order001")
    order_system.place_order("Order002")
    order_system.place_order("Order003")
    order_system.display_pending_orders()
    order_system.process_order()
    order_system.process_order()
    order_system.process_order()
    order_system.process_order()  # Attempt to process when queue is empty
```

Output:

```
C:\Users\acera\Desktop\Btech_3_2\AI Assistant coding>python
Order Order001 placed.
Order Order002 placed.
Order Order003 placed.
Pending orders:
Order001
Order002
Order003
Processing order Order001.
Processing order Order002.
Processing order Order003.
No orders to process.
```

Observation:

The Order Processing System correctly follows the FIFO principle, ensuring fairness in handling customer orders. The Queue data structure was chosen because it processes elements in the exact order they are inserted. This makes it the most suitable and logical structure for managing execution.